

Design and Analysis of Algorithms

Divide and Conquer II

Dr. Golzari

University of Hormozgan

Fall 1405

Logarithmic Power

Question: How Fast Is the Obvious Way?

$$a^n = \underbrace{a \cdot a \cdots a}_{n \text{ times}} \implies n - 1 \text{ multiplications.}$$

Question: How Fast Is the Obvious Way?

$$a^n = \underbrace{a \cdot a \cdots a}_{n \text{ times}} \implies n - 1 \text{ multiplications.}$$

- $n = 10^6 \Rightarrow$ a million multiplications.

Question: How Fast Is the Obvious Way?

$$a^n = \underbrace{a \cdot a \cdots a}_{n \text{ times}} \implies n - 1 \text{ multiplications.}$$

- $n = 10^6 \Rightarrow$ a million multiplications.
- **Question:** can we shrink the *exponent* drastically, instead of stepping down by one?

Observation: Squaring Jumps

$$a^{16} = \left(\left(\left(a^2 \right)^2 \right)^2 \right)^2$$

Observation: Squaring Jumps

$$a^{16} = \left(\left(\left(a^2 \right)^2 \right)^2 \right)^2$$

- 4 squarings instead of 15 multiplications.

Observation: Squaring Jumps

$$a^{16} = \left(\left((a^2)^2 \right)^2 \right)^2$$

- 4 squarings instead of 15 multiplications.
- **Question:** does the trick survive *odd* exponents?

Answer: Two Identities

$$a^{2k} = (a^k)^2 \qquad a^{2k+1} = (a^k)^2 \cdot a$$

Answer: Two Identities

$$a^{2k} = (a^k)^2 \qquad a^{2k+1} = (a^k)^2 \cdot a$$

- Compute a^k **once**, square it, maybe one extra $\cdot a$.

Two recursive calls:

$$a^n = a^{n/2} \cdot a^{n/2} \quad \implies \quad T(n) = 2T(n/2) + 1 = \Theta(n)$$

Two recursive calls:

$$a^n = a^{n/2} \cdot a^{n/2} \quad \implies \quad T(n) = 2T(n/2) + 1 = \Theta(n)$$

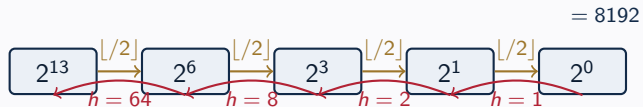
- No gain – the half power must be computed **once** and reused.

```
1: Power( $a$ ,  $n$ ):  
2: if  $n = 0$  then  
3:   return 1  
4:  $h \leftarrow \text{Power}(a, \lfloor n/2 \rfloor)$   
5: return  $h \cdot h$  (and  $\cdot a$  if  $n$  is odd)
```

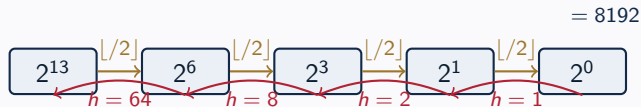
```
1: Power( $a$ ,  $n$ ):  
2: if  $n = 0$  then  
3:   return 1  
4:  $h \leftarrow \mathbf{Power}(a, \lfloor n/2 \rfloor)$   
5: return  $h \cdot h$  (and  $\cdot a$  if  $n$  is odd)
```

$$T(n) = T(n/2) + \Theta(1) = \Theta(\log n)$$

Example: Trace of 2^{13}



Example: Trace of 2^{13}



- 5 levels instead of 12 multiplications.

Example: The Binary View of $13 = 1101_2$

Read the bits left to right; start with $h = a$. Each bit: **square**, and one extra $\cdot a$ if the bit is 1.

Example: The Binary View of $13 = 1101_2$

Read the bits left to right; start with $h = a$. Each bit: **square**, and one extra $\cdot a$ if the bit is 1.

bit	operation	h	power
—	start	2	2^1

Example: The Binary View of $13 = 1101_2$

Read the bits left to right; start with $h = a$. Each bit: **square**, and one extra $\cdot a$ if the bit is 1.

bit	operation	h	power
—	start	2	2^1
1	square, $\cdot a$	8	2^3

Example: The Binary View of $13 = 1101_2$

Read the bits left to right; start with $h = a$. Each bit: **square**, and one extra $\cdot a$ if the bit is 1.

bit	operation	h	power
—	start	2	2^1
1	square, $\cdot a$	8	2^3
0	square	64	2^6

Example: The Binary View of $13 = 1101_2$

Read the bits left to right; start with $h = a$. Each bit: **square**, and one extra $\cdot a$ if the bit is 1.

bit	operation	h	power
—	start	2	2^1
1	square, $\cdot a$	8	2^3
0	square	64	2^6
1	square, $\cdot a$	8192	2^{13}

Example: The Binary View of $13 = 1101_2$

Read the bits left to right; start with $h = a$. Each bit: **square**, and one extra $\cdot a$ if the bit is 1.

bit	operation	h	power
—	start	2	2^1
1	square, $\cdot a$	8	2^3
0	square	64	2^6
1	square, $\cdot a$	8192	2^{13}

- 3 squarings + 3 multiplications = 6 (naive: 12).

Example: Powers with a Modulus

$$(x \cdot y) \bmod m = ((x \bmod m)(y \bmod m)) \bmod m$$

Example: Powers with a Modulus

$$(x \cdot y) \bmod m = ((x \bmod m)(y \bmod m)) \bmod m$$

$$3^6 = 729 \equiv 1 \pmod{7} \implies 3^{20} = (3^6)^3 \cdot 3^2 \equiv 1 \cdot 2 = 2 \pmod{7}$$

Example: Powers with a Modulus

$$(x \cdot y) \bmod m = ((x \bmod m)(y \bmod m)) \bmod m$$

$$3^6 = 729 \equiv 1 \pmod{7} \implies 3^{20} = (3^6)^3 \cdot 3^2 \equiv 1 \cdot 2 = 2 \pmod{7}$$

- Fast power + this rule = the exponentiation loop inside RSA.

Logarithmic Fibonacci

Question: Can We Reach F_n Without Walking There?

$$F_n = F_{n-1} + F_{n-2}$$

Question: Can We Reach F_n Without Walking There?

$$F_n = F_{n-1} + F_{n-2}$$

- Naive recursion: $\Theta(\varphi^n)$ calls ($\varphi \approx 1.618$) – explodes.

Question: Can We Reach F_n Without Walking There?

$$F_n = F_{n-1} + F_{n-2}$$

- Naive recursion: $\Theta(\varphi^n)$ calls ($\varphi \approx 1.618$) – explodes.
- Simple loop: n additions – still slow for $n = 10^6$.

Question: Can We Reach F_n Without Walking There?

$$F_n = F_{n-1} + F_{n-2}$$

- Naive recursion: $\Theta(\varphi^n)$ calls ($\varphi \approx 1.618$) – explodes.
- Simple loop: n additions – still slow for $n = 10^6$.
- **Idea:** turn Fibonacci's *addition* into *multiplication* – then reuse fast power.

Fibonacci Is a Matrix Power

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

Fibonacci Is a Matrix Power

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

- Proof: one induction – multiply by the matrix once and use $F_{n+1} = F_n + F_{n-1}$.

Fibonacci Is a Matrix Power

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

- Proof: one induction – multiply by the matrix once and use $F_{n+1} = F_n + F_{n-1}$.
- So F_n is just one entry of a matrix power.

The Same Power Algorithm

Power(M , n) with base case $M^0 = I$.

The Same Power Algorithm

Power(M, n) with base case $M^0 = I$.

- One squaring of a 2×2 matrix = 8 scalar multiplications.

The Same Power Algorithm

Power(M, n) with base case $M^0 = I$.

- One squaring of a 2×2 matrix = 8 scalar multiplications.
- $\Theta(\log n)$ matrix multiplications $\Rightarrow F_n$ in $\Theta(\log n)$.

Example: Building M^5 by Squaring

$$M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

Example: Building M^5 by Squaring

$$M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

$$M^2 = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$$

Example: Building M^5 by Squaring

$$M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

$$M^2 = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$$

$$M^4 = (M^2)^2 = \begin{pmatrix} 5 & 3 \\ 3 & 2 \end{pmatrix} \quad M^5 = M^4 \cdot M = \begin{pmatrix} 8 & 5 \\ 5 & 3 \end{pmatrix}$$

Example: Building M^5 by Squaring

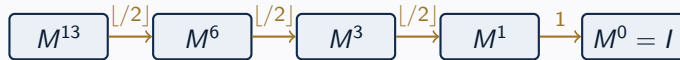
$$M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

$$M^2 = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$$

$$M^4 = (M^2)^2 = \begin{pmatrix} 5 & 3 \\ 3 & 2 \end{pmatrix} \quad M^5 = M^4 \cdot M = \begin{pmatrix} 8 & 5 \\ 5 & 3 \end{pmatrix}$$

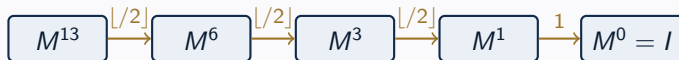
- Entry (1, 2): $F_5 = 5$. ✓

Example: Trace of M^{13}



returning: square (+ $\cdot M$ if odd)

Example: Trace of M^{13}



returning: square (+ $\cdot M$ if odd)

- 4 matrix multiplications; entry $(1, 2)$ gives $F_{13} = 233$.

Multiplying Big Integers

Question: Can We Beat n^2 Digit-Products?

schoolbook: every digit $\times$ every digit $\implies n^2$ products.

Question: Can We Beat n^2 Digit-Products?

schoolbook: every digit $\times$ every digit $\implies n^2$ products.

- RSA keys: ≈ 600 digits — n^2 starts to hurt.

Question: Can We Beat n^2 Digit-Products?

schoolbook: every digit $\times$ every digit $\implies n^2$ products.

- RSA keys: ≈ 600 digits — n^2 starts to hurt.
- **Question:** where exactly is the waste?

Step 1: Split in Half

$$x = x_1 B^m + x_0, \quad y = y_1 B^m + y_0 \quad (m = n/2)$$

Step 1: Split in Half

$$x = x_1 B^m + x_0, \quad y = y_1 B^m + y_0 \quad (m = n/2)$$

$$xy = x_1 y_1 B^{2m} + (x_1 y_0 + x_0 y_1) B^m + x_0 y_0$$

Step 1: Split in Half

$$x = x_1 B^m + x_0, \quad y = y_1 B^m + y_0 \quad (m = n/2)$$

$$xy = x_1 y_1 B^{2m} + (x_1 y_0 + x_0 y_1) B^m + x_0 y_0$$

- **Four** half-size multiplications: $T(n) = 4T(n/2) + \Theta(n)$.

Step 1: Split in Half

$$x = x_1 B^m + x_0, \quad y = y_1 B^m + y_0 \quad (m = n/2)$$

$$xy = x_1 y_1 B^{2m} + (x_1 y_0 + x_0 y_1) B^m + x_0 y_0$$

- **Four** half-size multiplications: $T(n) = 4T(n/2) + \Theta(n)$.
- $4 = 2^2$: still $\Theta(n^2)$ – no gain.

Step 2: The Four Products Are Not Independent

$$(x_1 + x_0)(y_1 + y_0) = \underbrace{x_1 y_1}_{P_1} + \underbrace{x_0 y_0}_{P_2} + \underbrace{(x_1 y_0 + x_0 y_1)}_{\text{middle}}$$

Step 2: The Four Products Are Not Independent

$$(x_1 + x_0)(y_1 + y_0) = \underbrace{x_1 y_1}_{P_1} + \underbrace{x_0 y_0}_{P_2} + \underbrace{(x_1 y_0 + x_0 y_1)}_{\text{middle}}$$

$$\text{middle} = P_3 - P_1 - P_2, \quad P_3 = (x_1 + x_0)(y_1 + y_0)$$

Step 2: The Four Products Are Not Independent

$$(x_1 + x_0)(y_1 + y_0) = \underbrace{x_1 y_1}_{P_1} + \underbrace{x_0 y_0}_{P_2} + \underbrace{(x_1 y_0 + x_0 y_1)}_{\text{middle}}$$

$$\text{middle} = P_3 - P_1 - P_2, \quad P_3 = (x_1 + x_0)(y_1 + y_0)$$

- Only **3** multiplications + cheap additions:

$$T(n) = 3 T(n/2) + \Theta(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$$

Worked Example: 1234×5678 (Step 1)

Base $B = 100$, $m = 2$: $x_1 = 12$, $x_0 = 34$, $y_1 = 56$, $y_0 = 78$.

Worked Example: 1234×5678 (Step 1)

Base $B = 100$, $m = 2$: $x_1 = 12$, $x_0 = 34$, $y_1 = 56$, $y_0 = 78$.

$$P_1 = 12 \cdot 56 = 672 \quad P_2 = 34 \cdot 78 = 2652$$

Worked Example: 1234×5678 (Step 1)

Base $B = 100$, $m = 2$: $x_1 = 12$, $x_0 = 34$, $y_1 = 56$, $y_0 = 78$.

$$P_1 = 12 \cdot 56 = 672 \quad P_2 = 34 \cdot 78 = 2652$$

$$P_3 = (12 + 34)(56 + 78) = 46 \cdot 134 = 6164$$

Worked Example: 1234×5678 (Step 2)

$$\text{middle} = P_3 - P_1 - P_2 = 6164 - 672 - 2652 = 2840$$

Worked Example: 1234×5678 (Step 2)

$$\text{middle} = P_3 - P_1 - P_2 = 6164 - 672 - 2652 = 2840$$

$$xy = 672 \cdot 100^2 + 2840 \cdot 100 + 2652 = 7,006,652$$

Worked Example: 1234×5678 (Step 2)

$$\text{middle} = P_3 - P_1 - P_2 = 6164 - 672 - 2652 = 2840$$

$$xy = 672 \cdot 100^2 + 2840 \cdot 100 + 2652 = 7,006,652$$

- Check: $1234 \cdot 5678 = 7,006,652$. ✓

Worked Example: 1234×5678 (Step 2)

$$\text{middle} = P_3 - P_1 - P_2 = 6164 - 672 - 2652 = 2840$$

$$xy = 672 \cdot 100^2 + 2840 \cdot 100 + 2652 = 7,006,652$$

- Check: $1234 \cdot 5678 = 7,006,652$. ✓
- Only **3** small multiplications instead of 4.

digits n	schoolbook n^2	Karatsuba $n^{1.585}$
100	10^4	$\approx 1.5 \cdot 10^3$

digits n	schoolbook n^2	Karatsuba $n^{1.585}$
100	10^4	$\approx 1.5 \cdot 10^3$
1,000	10^6	$\approx 6 \cdot 10^4$

digits n	schoolbook n^2	Karatsuba $n^{1.585}$
100	10^4	$\approx 1.5 \cdot 10^3$
1,000	10^6	$\approx 6 \cdot 10^4$
10^6	10^{12}	$\approx 3 \cdot 10^9$

digits n	schoolbook n^2	Karatsuba $n^{1.585}$
100	10^4	$\approx 1.5 \cdot 10^3$
1,000	10^6	$\approx 6 \cdot 10^4$
10^6	10^{12}	$\approx 3 \cdot 10^9$

- Python's big integers and GMP multiply exactly this way.

Wrap-Up

Key Takeaways

- Fast power: one call per level $\Rightarrow \Theta(\log n)$ – never two calls.

Key Takeaways

- Fast power: one call per level $\Rightarrow \Theta(\log n)$ – never two calls.
- Same algorithm on matrices (base case I) $\Rightarrow$ Fibonacci in $\Theta(\log n)$.

Key Takeaways

- Fast power: one call per level $\Rightarrow \Theta(\log n)$ – never two calls.
- Same algorithm on matrices (base case I) $\Rightarrow$ Fibonacci in $\Theta(\log n)$.
- Karatsuba: $4 \rightarrow 3$ half-size products $\Rightarrow n^{1.585}$.

Key Takeaways

- Fast power: one call per level $\Rightarrow \Theta(\log n)$ – never two calls.
- Same algorithm on matrices (base case I) $\Rightarrow$ Fibonacci in $\Theta(\log n)$.
- Karatsuba: $4 \rightarrow 3$ half-size products $\Rightarrow n^{1.585}$.
- Halving beats stepping down by one: $\log n$ versus n .

1. Compute $7^{100} \bmod 13$ by hand with fast power; count your multiplications.
2. Prove the Fibonacci matrix identity by induction.
3. Compute M^{10} by squaring, read off F_{10} , and count the scalar multiplications.
4. Apply Karatsuba to 2394×8176 (base 100), following the worked example.
5. **(Challenge)** Fast doubling: $F_{2k} = F_k(2F_{k+1} - F_k)$, $F_{2k+1} = F_{k+1}^2 + F_k^2$. How many multiplications per level – fewer than the matrix method?